

1. Employee Salary Update

Scenario: Write a Java program to update the salary of an employee in a class **Employee**. The class should have private attributes **id**, **name**, and **salary**. Include a method **updateSalary** that takes a percentage as input and increases the salary by that percentage. Use appropriate constructors and methods to manage the employee's data.

- **Sample Input:**
Employee emp = new Employee(1, "John Doe", 50000);
emp.updateSalary(10); // Increase salary by 10%
- **Sample Output:**
New Salary: 55000

2. Book Management

Scenario: Create a Java class **Book** with attributes **title**, **author**, and **status** (available or not). Write a method **toggleStatus** to change the status of the book (if available, make it unavailable, and vice versa). Include a constructor to initialize the book's details.

- **Sample Input:**
Book book = new Book("Java Basics", "Jane Doe", true);
book.toggleStatus();
- **Sample Output:**
Status: unavailable

3. Basic Calculator

Scenario: Design a class **Calculator** that can perform basic arithmetic operations (addition, subtraction, multiplication, division). The class should have methods for each operation that take two numbers as arguments and return the result.

- **Sample Input:**
Calculator calc = new Calculator();
calc.add(5, 3);
calc.divide(10, 2);
- **Sample Output:**
Addition Result: 8
Division Result: 5

4. Array Manipulator

Scenario: Implement a class **ArrayManipulator** that has a method **findMax** to find the maximum number in an array of integers and a method **findMin** to find the minimum number.

- **Sample Input:**
`int[] numbers = {1, 3, 5, 7, 9, 2, 4, 6, 8};`
`ArrayManipulator.findMax(numbers);`
`ArrayManipulator.findMin(numbers);`
- **Sample Output:**
`Max: 9`
`Min: 1`

5. String Reverser

Scenario: Write a Java class **StringReverser** with a static method **reverse** that takes a string as input and returns the reversed string.

- **Sample Input:**
`StringReverser.reverse("hello");`
- **Sample Output:**
`olleh`

6. Prime Number Checker

Scenario: Create a class **PrimeChecker** with a method **isPrime** that takes an integer as input and returns **true** if the number is prime and **false** otherwise.

- **Sample Input:**
`PrimeChecker.isPrime(11);`
`PrimeChecker.isPrime(4);`
- **Sample Output:**
`true`
`false`

7. Student Grade Calculator

Scenario: Develop a class **Student** with private attributes for **name**, **rollNo**, and a **grades** array. Include a method **calculateAverage** that computes the average grade.

- **Sample Input:**
Student student = new Student("Alex", 101, new int[]{85, 92, 88});
student.calculateAverage();
- **Sample Output:**
Average Grade: 88.33

8. Matrix Addition

Scenario: Implement a class **Matrix** with a static method **addMatrices** that takes two 2D arrays representing matrices and returns their sum as a new 2D array.

- **Sample Input:**
int[][] matrix1 = {{1, 2}, {3, 4}};
int[][] matrix2 = {{2, 3}, {4, 5}};
Matrix.addMatrices(matrix1, matrix2);
- **Sample Output:**
{{3, 5}, {7, 9}}

Each of these questions requires students to apply various OOP concepts, manipulate data types, and use control structures effectively, reflecting both the breadth and depth of the syllabus covered.

9. Password Validator

Scenario: Create a class **PasswordValidator** with a method **isValid** that checks if a given password meets a set of criteria (e.g., at least 8 characters, contains at least one digit). The method should return **true** if the password is valid, and **false** otherwise.

- **Sample Input:**
PasswordValidator.isValid("Password123");
PasswordValidator.isValid("pass");
- **Sample Output:**
true
false

10. Custom String Concatenation

Scenario: Implement a class **StringCombiner** with a static method **combine** that takes an array of strings and concatenates them into a single string, separated by a custom delimiter provided as a second argument.

- **Sample Input:**
`StringCombiner.combine(new String[]{"Java", "Python", "C++"}, ";");`
- **Sample Output:**
`Java;Python;C++`

11. Fibonacci Series Generator

Scenario: Design a class **FibonacciGenerator** with a static method **generate** that takes an integer **n** and prints the first **n** Fibonacci numbers.

- **Sample Input:**
`FibonacciGenerator.generate(5);`
- **Sample Output:**
`0 1 1 2 3`

12. Shopping Cart Total Calculator

Scenario: Create a class **ShoppingCart** with a method **calculateTotal** that calculates the total price of items in a shopping cart. Assume each item is represented as a **Product** object with **name** and **price** attributes.

- **Sample Input:**
`ShoppingCart cart = new ShoppingCart();
cart.addProduct(new Product("Apple", 1.0));
cart.addProduct(new Product("Banana", 2.0));
cart.calculateTotal();`
- **Sample Output:**
`Total: 3.0`

13. Circle Area Calculator

Scenario: Develop a class **Circle** with a constructor that takes the radius as an input and a method **calculateArea** that calculates and returns the area of the circle.

- **Sample Input:**
`Circle circle = new Circle(5);
circle.calculateArea();`
- **Sample Output:**
`Area: 78.54`

14. Unique Number Finder

Scenario: Write a Java class **UniqueNumbers** with a method **findUnique** that takes an array of integers and returns an array containing only the unique numbers, in the order they appeared.

- **Sample Input:**
`UniqueNumbers.findUnique(new int[]{1, 2, 3, 2, 4, 1});`
- **Sample Output:**
`[1, 2, 3, 4]`

15. Email Extractor

Scenario: Implement a class **EmailExtractor** with a static method **extract** that takes a string containing text and extracts all email addresses into an array of strings.

- **Sample Input:**
`EmailExtractor.extract("Contact us at support@example.com or feedback@example.net.");`
- **Sample Output:**
`["support@example.com", "feedback@example.net"]`

16. Bank Account Simulator

Scenario: Design a class **BankAccount** with methods to deposit and withdraw money, along with an attribute to keep track of the balance. Include checks to ensure withdrawals do not exceed the available balance.

- **Sample Input:**
`BankAccount account = new BankAccount(100);
account.deposit(50);
account.withdraw(120);
account.withdraw(30);`
- **Sample Output:**
`Withdrawal successful: 120
Insufficient balance for withdrawal: 30`

17. Rectangle Perimeter Calculator

Scenario: Create a class **Rectangle** with private attributes **length** and **width**. Write a method **calculatePerimeter** that calculates and returns the perimeter of the rectangle.

- **Sample Input:**
Rectangle rect = new Rectangle(5, 10);
rect.calculatePerimeter();
- **Sample Output:**
Perimeter: 30

18. Student Database Query

Scenario: Suppose you have a class **StudentDatabase** that stores student records. Each record is a **Student** object with **name**, **rollNumber**, and **department**. Write a method **findByDepartment** that returns a list of students belonging to a specified department.

- **Sample Input:**
StudentDatabase db = new StudentDatabase();
db.findByDepartment("Computer Science");
- **Sample Output:**
["John Doe", "Jane Smith"]

19. Stack Implementation

Scenario: Implement a simple class **Stack** with methods **push**, **pop**, and **peek**. Ensure your implementation correctly handles underflow (popping from an empty stack) and overflow (pushing when the stack is at maximum capacity).

- **Sample Input:**
Stack stack = new Stack(3);
stack.push(1); stack.push(2); stack.push(3); stack.push(4); // Overflow
stack.pop(); stack.pop(); stack.pop(); stack.pop(); // Underflow
- **Sample Output:**
Overflow error
Underflow error

20. Temperature Converter

Scenario: Write a class **TemperatureConverter** with static methods **celsiusToFahrenheit** and **fahrenheitToCelsius** that convert temperatures between the two scales.

- **Sample Input:**
TemperatureConverter.celsiusToFahrenheit(0);
TemperatureConverter.fahrenheitToCelsius(32);

- **Sample Output:**
32.0
0.0